

# BUG'S DEBUG GARAGE

## FULL DEBUGGING CHEAT SHEET

Method in Motion Coding | Bug - Debug Gremlin

### Lesson 1 - What Is a Bug?

|                      |                                                                                              |
|----------------------|----------------------------------------------------------------------------------------------|
| <b>Bug</b>           | A gap between what code was intended to do and what it actually does.                        |
| <b>Syntax error</b>  | Invalid code structure - the code cannot run correctly until the syntax is fixed.            |
| <b>Runtime error</b> | A failure that happens after the code starts running.                                        |
| <b>Logic error</b>   | The code runs, but the result is wrong.                                                      |
| <b>console.log()</b> | Prints values to the browser console so you can inspect what the code knows.                 |
| <b>Bug history</b>   | The term "bug" predates 1947. The Harvard Mark II moth became the famous "actual bug" story. |

### Lesson 2 - Reading Errors

|                       |                                                                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ReferenceError</b> | Code tried to use a name that cannot be found where it is being used.                                                                            |
| <b>TypeError</b>      | A value is the wrong kind for the action being attempted.                                                                                        |
| <b>SyntaxError</b>    | The code structure is invalid.                                                                                                                   |
| <b>RangeError</b>     | A value is outside the allowed range for an operation.                                                                                           |
| <b>Call stack</b>     | Start near the top: the error and first relevant stack line usually point closest to where the problem surfaced; lines below show the call path. |
| <b>First 3 checks</b> | Error type, exact message, then line number or first relevant location.                                                                          |

## Lesson 3 - Logic Tracing

|                           |                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Logic tracing</b>      | Step through code in order while tracking every important variable and condition.                                     |
| <b>Boundary condition</b> | The first or last allowed value - the edges where many loop and condition bugs hide.                                  |
| <b>Off-by-one</b>         | One too many or one too few. Check <code>&lt;</code> vs <code>&lt;=</code> , starting values, and zero-based indexes. |
| <b>Infinite loop</b>      | A loop whose exit condition never becomes false. Stop/refresh, then find what should change so it can end.            |
| <b>Rubber duck</b>        | Explain each line out loud so skipped assumptions become easier to notice.                                            |

## Lesson 4 - Advanced Debugging

|                             |                                                                                                        |
|-----------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Divide &amp; conquer</b> | Test smaller sections until the bug's hiding place is narrowed down.                                   |
| <b>Minimal reproduction</b> | Reduce the code to the smallest version that still shows the bug.                                      |
| <b>try/catch</b>            | try runs risky code; catch(err) receives an error if the try block throws one.                         |
| <b>Defensive check</b>      | if (val && typeof val === "string") checks that a value exists first, then checks that it is a string. |
| <b>Debug mindset</b>        | Never assume - verify. Use evidence like logs, types, outputs, and small tests.                        |

## Symbols Reference

|                           |                                                                                           |
|---------------------------|-------------------------------------------------------------------------------------------|
| <code>()</code>           | Parentheses - call a function or hold the inputs/arguments passed to it.                  |
| <code>.</code>            | Dot - accesses a property or method that belongs to another value.                        |
| <code>=</code>            | Equals sign - assigns a value.                                                            |
| <code>&lt; / &lt;=</code> | Less than / less than or equal - common places to check for boundary and off-by-one bugs. |